

Proximity risk after go-arounds and touch-and-goes — terminal commands

Every command and option of run_proximity.py with examples. Run everything from the repository root with the virtual environment active. Method: FRAMEWORK.md.

1. Set-up (once per machine)

<pre>git clone \ https://github.com/sponbob3/CollisionRiskAlgo.git cd CollisionRiskAlgo</pre>	Get the repository.
<pre>python3.12 --version</pre>	Use Python 3.12 (3.11 also works). If the command is not found, install Python 3.12 from python.org (or brew install python@3.12) and open a new terminal. Other versions can stay installed: macOS's own python3 may be too old (3.9), and Python 3.13+ is too new for some of the traffic library's dependencies.
<pre>python3.12 -m venv .venv source .venv/bin/activate # Windows: .venv\Scripts\activate python -m pip install --upgrade pip python -m pip install -r requirements.txt</pre>	Create the virtual environment and install every dependency (the same packages as the go-around pipeline plus scipy, statsmodels, pyproj, airportsdata and pytest). 'python -m pip' works even where a bare 'pip' command is not found.
<pre>source .venv/bin/activate</pre>	Run this in every NEW terminal window, inside the repository, before any other command; the prompt then starts with (.venv).
<pre>git pull</pre>	Update to the latest version later (run inside the repository).

2. Data

<pre>mkdir -p datasets/KMCO_2025Q1 cp -R /path/to/kmco_data/* \ datasets/KMCO_2025Q1/</pre>	One file per day (.parquet preferred, .csv accepted) of OpenSky-style state vectors for ALL traffic around the airport, either directly in the folder or in subfolders (e.g. one per month). Columns: timestamp, icao24, callsign, latitude, longitude, altitude, geoaltitude, vertical_rate, groundspeed, track, onground; a position-time column (last_position) is used when present to drop rows that repeat an old position. The folder name is <ICAO>_<label>; the prefix selects airports/<ICAO>.yaml. datasets/ is gitignored.
<pre>python run_proximity.py check-data KMCO_2025Q1 python run_proximity.py check-data \ KMCO_2025Q1 --limit 5</pre>	Data audit only: days found, columns, per-day message and aircraft counts, gaps and receiver outages, altitude references, and whether the volume meets the baseline minimum (28 days, 90 recommended). Writes output/KMCO/proximity_risk/check_data_KMCO_2025Q1.csv.

3. Airport profile (automatic)

<code>python run_proximity.py KMC0_2025Q1</code>	Nothing to do by hand: when airports/KMCO.yaml does not exist, the first run (or check-data) creates it - runways and elevation from the OurAirports database (downloaded once, needs internet), timezone from the coordinates, preset and assume_arrivals_dataset detected from the data, the ADS-B coverage floor near the field measured from the data, and the airspace block (field elevation + 4,000 ft, checked against the data every run). Each detected value is printed and written into the file with its evidence.
<code>python run_proximity.py new-airport KMCO \ KMCO_2025Q1</code>	Create the profile without running the analysis (the dataset name is optional; without it the preset comes from the airport type).
<code>open airports/KMCO.yaml</code>	The one place to change any airport setting (timezone, preset, terrain, assume_arrivals_dataset, ceiling, flows, go-around overrides). It is never overwritten; delete it to regenerate.

4. Running the analysis

<code>python run_proximity.py KMC0_2025Q1</code>	The complete run: go-around detection -> output/KMCO/goaround/run_NN/, then baseline (built or reused), encounters, window metrics, every event, sensitivity sweep, tables, figures, per-event plots and summary.pdf -> output/KMCO/proximity_risk/run_NN/ (same NN).
<code>python run_proximity.py KMC0_2025Q1 --limit \ 5 --no-sensitivity --no-plots</code>	Quick test on the first 5 days without the sweep or figures.
<code>python run_proximity.py KMC0_2025Q1 \ --workers 4</code>	Parallel day processing (default: CPU count - 1).
<code>python run_proximity.py KMC0_2025Q1 \ --goaround-report --calibrate</code>	Also write the go-around stage's figures, narrative report and PDF, and its plateau-duration calibration histogram.
<code>python run_proximity.py KMC0_2025Q1 --window 5 python run_proximity.py KMC0_2025Q1 \ --window 15</code>	Pre/post window length in minutes (default 10; the sweep runs 5 and 15 anyway).
<code>python run_proximity.py KMC0_2025Q1 \ --include-ambiguous</code>	Events are go-arounds and touch-and-goes (each keeps its own name; results are also given per type). This adds ga_ambiguous go-arounds.
<code>python run_proximity.py KMC0_2025Q1 \ --no-event-plots</code>	Skip the one-PNG-per-event plots only (figures and summary.pdf are still produced).

5. Reusing runs

<code>python run_proximity.py KMC0_2025Q1 \ --goaround-run 3</code>	Reuse output/KMCO/goaround/run_03 instead of re-running go-around detection (the proximity run still gets a new NN).
---	--

<code>ls output/KMCO/proximity_risk/cache/2025Q1/</code>	Per-day pairwise caches (parameter-hashed). They are reused automatically by every run and sweep; delete the folder to force recomputation.
--	---

6. Baselines

<code>python run_proximity.py KMCO_2025Q1 \</code> <code>--baseline-only</code>	Build (or reuse) and validate the baseline, write <code>baseline_report.pdf</code> , and stop. Stored in <code>output/KMCO/proximity_risk/baseline/2025Q1/</code> (<code>baseline.json</code> , <code>baseline_windows.parquet</code> , <code>error_model.csv</code> , <code>phase1_removed.csv</code> , <code>validation/</code>).
<code>python run_proximity.py KMCO_2025Q1 \</code> <code>--rebuild-baseline</code>	Force a rebuild even if the stored baseline matches the data and parameters (it is rebuilt automatically when either changes).
<code>python run_proximity.py KMCO_2025Q2 \</code> <code>--baseline-from KMCO_2025Q1</code>	Score another dataset's events against the stored KMCO_2025Q1 baseline (same airport).

7. Checks and tests

<code>python -m pytest</code>	The whole test suite (analytic intervals vs brute force, missed passes, scenarios, pruning, probability vs Monte Carlo, baseline calibration, injected effect, go-around parity, traffic cross-check). About two minutes.
<code>python -m pytest -q -m 'not slow'</code>	Skip the end-to-end synthetic runs.
<code>python -m proximity_pipeline.validate_cpa \</code> <code>KMCO_2025Q1 20250108 20250115</code>	Cross-check closest-approach distances against the traffic library on the named days.
<code>python tools/make_synthetic_dataset.py \</code> <code>KBNA_synth --days 6</code> <code>python run_proximity.py KBNA_synth --limit 3</code>	Synthetic traffic for a smoke test without real data (<code>--intruder-rate</code> and <code>--post-ga-irr</code> inject a known effect).
<code>python tools/make_terminal_commands_pdf.py</code>	Regenerate this PDF after changing the CLI.

8. Where things land

<code>output/KMCO/goaround/run_NN/</code>	Go-around pipeline outputs (<code>all_approaches.csv</code> , <code>go_around_events.csv</code> , <code>summary.pdf</code> , <code>plots/</code> , <code>run_config.txt</code>).
<code>output/KMCO/proximity_risk/run_NN/</code>	<code>run_config.txt</code> , <code>ceiling_check.csv</code> , <code>data_quality.csv</code> , <code>window_metrics.parquet</code> , <code>encounters.parquet/</code> . <code>csv</code> , <code>go_around_risk.csv</code> (+ <code>_involved</code>), <code>results_primary.csv</code> , <code>equilibrium.csv</code> , <code>epoch.csv</code> , <code>sensitivity.csv</code> , <code>figures/</code> , <code>events/</code> , <code>summary.pdf</code> .

Options summary: `--goaround-run N`, `--rebuild-baseline`, `--baseline-only`, `--baseline-from ICAO_label`, `--window MIN`, `--include-ambiguous`, `--no-sensitivity`, `--no-plots`, `--no-event-plots`, `--goaround-report`, `--calibrate`, `--limit N`, `--workers N`.
Sub-commands: `new-airport ICAO`, `check-data ICAO_label [--limit N]`.